中間テスト 完全突破対策

プログラミング実習

「分かる」を「何も見ずに書ける」に変える。

本編： の重点 型を攻略
ポインタで複数の値を返す
構造体へのポインタ
配列・乱数・度数分布
次元配列・行列演算

この 冊でやること

要点と定番コード 間違い発見 白紙再現 分模試 回 解答・境界値チェック。読むだけではなく、必ず手を動かして使う。

対象範囲と出典について

本編は既存アプリ内の 対策情報と ～ の重点項目に基づく。正式な試験予告 自体は今回の添付にはないため、日程・持込条件・採点方法は最新の教員指示で確認する。

添付「 動的割当て 」「 」は補講（ ～ ページ）の根拠として使用。 を の確定出題範囲とは扱わない。模試・小問は本書オリジナルであり、実際の出題を保証しない。

到達目標： 型を白紙から実装し、コンパイル・実行・ ・境界値検証まで自力で完了する。

攻略ルートと目次

まず自分の弱点を　分で診断

　　と　　の違いを説明できる

　　を迷わず書ける

カウンタ配列を毎回　に初期化できる

転置と行列積を区別できる

　　を複数指定して　　し、出力を　　できる

目次

試験の進め方・コマンド　　ページ

型・入出力・　　ページ

ポインタの定番　　ページ

構造体ポインタ　　ページ

配列と乱数・度数分布　　ページ

行列の基本 / 転置・積　　〜　ページ

失点パターン・即答ドリル　　〜　　ページ

模試　：問題 / 解答　　〜　　ページ

模試　：問題 / 解答　　〜　　ページ

　　動的割当て補講・演習　　〜　　ページ

前日・当日チェック　　ページ

残り時間別の勉強法

あと　日：　日目に　型を再現、　日目に模試　、　日目に模試　と解き直し。

あと　日：　〜　　ページ　　模試　　間違えた型だけ再現　　ページ。

あと　時間：　〜　・　〜　　ページを優先。新しい応用より、既知の型を確実に。

合格を狙う順序は「得意な　問を完成　　問目　　最終確認」。　問すべてを未完成にするより、完成・検証済みの問題を確保する。

本番は「検証まで」が　問

アプリ内の　　案内では　　分・　問　　点・　　点以上で合格、持込不可・部分点なしとされる。最新の公式案内が異なる場合はそちらを優先する。

分の配分例

- 〜　分：全問を読む。実装ファイル・関数・制約・得意順をメモ。
- 〜　　分：最も確実な　問を完成させ、　　まで実施。
- 〜　　分：　問目を完成させ、隠し入力を想定して再確認。
- 〜　　分：　問目。行き詰まったら動く問題の保全を優先。
- 〜　　分：保存名・場所・コンパイル・出力を最後に確認。

必ず指が覚えるコマンド

```
gcc -std=c11 -Wall -Wextra main.c sub.c -lm -o run
./run < in.txt > out.txt
diff -Bw out.txt exp.txt

# Separate compilation
gcc -c main.c
gcc -c sub.c
gcc main.o sub.o -lm -o run
```

数式関数を使わないときは　　不要。　　で実行名を統一すると環境差で迷わない。配布が指定するコマンドがあれば優先する。

エラーの分類

コンパイル：文法・型・宣言。最初のエラーから直す。
リンク：　　関数の実装や　、　　の指定を確認。
実行：落ちる　　の　、添字、　　、未初期化を確認。
出力：　　が出る　数式、書式、余計な　　を確認。

　　が失敗したら実行しない。前の実行ファイルが残り、直ったと錯覚することがある。　　が無言でも、全入力で正しい証明にはならない。

型・入出力・ の最小セット

と では の指定が異なる

```
int n;
double x;
char name[16];
scanf("%d", &n);
scanf("%lf", &x);
scanf("%15s", name);
printf("%d %.2f %s\n", n, x, name);
fprintf(stderr, "n = ");
```

数値の には書込先の番地が必要。文字配列 は式中で先頭要素へのポインタになるため、 では通常 を付けない。 なら終端文字込みで バイト以上必要。

と読み込み失敗を区別する

```
int result;
while ((result = scanf("%d", &n)) == 1) {
    printf("%d\n", n);
}
if (result != EOF) {
    fprintf(stderr, "invalid input\n");
}
```

は代入成功数を返す。 個読むなら成功は 。 だけの判定は、資料のように正しい入力だけを前提にした形では使われるが、不正な文字では を返すこともある。

整数演算・文字列の落とし穴

は 、 は 。整数同士の平均では で割る。
の は累乗ではなく排他的論理和。 乗は 。
文字列は ではなく で内容比較。
文字列コピーは代入ではなく 等。終端の空間を確保する。

```
必要ヘッダ： = 入出力 / = ・ ・ /
= ・ / = ・ ・ 。
```

型　：ポインタで複数の値を返す

値・番地・番地の中身を区別

```
int x = 7;
int *p = &x;
*p = 10;       /* x becomes 10 */
```

　は整数、　　は　の番地、　は番地を保持する変数、　　は指す先の整数。宣言の　と、式中の間接参照の　を区別する。

商と余り / 平均の定番

```
void divrem(int x, int y, int *q, int *r)
{
    *q = x / y;
    *r = x % y;
}
/* y != 0; call: divrem(x, y, &q, &r); */

void ave(double a, double b, double *pa, double *pm)
{
    *pa = (a + b) / 2.0;
    *pm = sqrt(a * b);
}
```

　　は　　とリンク時の　　が必要。相乗平均を実数で扱う入力条件を確認する。関数の目的が「値を書き戻す」なら、勝手に　　しない。

ポインタを別の関数に渡すとき

```
void swap_int(int *a, int *b)
{
    int tmp = *a;
    *a = *b;
    *b = tmp;
}
/* main: swap_int(&x, &y); */
/* inside a pointer function: swap_int(a, b); */
```

　　はポインタ自身を変更。　　は呼び出し元の値を変更。すでに　　である　に　を付けると　　になり、型が変わる。

型 ：構造体へのポインタ

構造体の値なら / ポインタなら

```
typedef struct {
    double x;
    double y;
} point_t;

int point_scan(point_t *p)
{
    int r = scanf("%lf%lf", &p->x, &p->y);
    if (r == EOF) return EOF;
    if (r != 2) exit(1);
    return 1;
}

double dist(point_t *a, point_t *b)
{
    double dx = a->x - b->x;
    double dy = a->y - b->y;
    return sqrt(dx * dx + dy * dy);
}
```

は と同じ。 に渡す はメンバー の番地。 全体の番地や、メンバーの値そのものではない。上記は ・ ・ が必要。

ヘッダと実装の分担

：型定義・関数プロトタイプ・必要なマクロ。
：ヘッダを して関数の本体を書く。
：ヘッダを して関数を呼ぶ。
同じ構造体の を に重ねて書かない。 も増やさない。

よく問われる応用

重心： 。 も同様。
価格変更： 。値渡しでは元の構造体は変わらない。
距離： の平方根。 は実数の絶対値、 は整数用。

戻り値の仕様を守る。「成功で 」「 なら 」「計算値を 」「出力用引数に格納」は別々の契約。

型　：配列・乱数・度数分布

配列引数は先頭要素へのポインタ

```
void fill(int a[], int n, int value)
{
    for (int i = 0; i < n; i++) a[i] = value;
}

void dice_throw(int d[], int n)
{
    for (int i = 0; i < n; i++)
        d[i] = rand() % 6 + 1;
}
```

呼び出しは　　。関数内の変更は元の配列に反映される。添字は　〜　　。関数内で　　から配列長を計算してはいけない。

個のサイコロ：和は　〜

```
void dice3_distr(int n, int d1[], int d2[],
                 int d3[], int count[])
{
    for (int s = 3; s <= 18; s++) count[s] = 0;
    for (int i = 0; i < n; i++) {
        int s = d1[i] + d2[i] + d3[i];
        count[s]++;
    }
}
```

この添字設計では　　まで使うので、呼び出し側は　　要素以上必要。和そのものを添字にする型と、　　を添字にする型を混同しない。

検証の目線

出現回数の合計は　か。　　ならすべて　か。
サイコロの各目は　〜　か。　　だけだと　〜　になる。
関数を　回呼んでも、前回のカウントが残らないか。
　　の有無・呼ぶ場所・　　を呼ぶ順序は配布　　の仕様に合わせる。

　　は授業の指定方式。厳密な確率の一様性や乱数列の再現性は実装に依存する。別・別ライブラリの列を同一だと決めつけない。

型 : 次元配列の基本

以下の は確保済みの最大列数、 は実際に処理する行・列数。前提は 。ヘッダで とプロトタイプを共有する。

読み込み・表示・和

```
#define N 8

void matrix_scan(int n, double A[N][N])
{
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%lf", &A[i][j]);
}

void matrix_print(int n, double A[N][N])
{
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
            printf(" %9.5f", A[i][j]);
        printf("\n");
    }
}

void matrix_sum(int n, double A[N][N],
                double B[N][N], double Y[N][N])
{
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            Y[i][j] = A[i][j] + B[i][j];
}
```

上の は正しい入力を前提とする授業用の最小形。頑健にするなら の返り値が か確認する。行の改行は内側ループが終わった位置。

とも宣言できるが、 とは同じ型ではない。通常の 次元配列の関数では列数 を揃える必要がある。

転置・行列ベクトル積・行列積

自分自身を転置：対角線の片側だけ交換

```
void matrix_trans1(int n, double A[N][N])
{
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            double t = A[i][j];
            A[i][j] = A[j][i];
            A[j][i] = t;
        }
    }
}
```

:　重ループ

```
void matrix_vector_prod(int n, double A[N][N],
                        double x[], double y[])
{
    for (int i = 0; i < n; i++) {
        y[i] = 0.0;
        for (int j = 0; j < n; j++)
            y[i] += A[i][j] * x[j];
    }
}
```

:　重ループ / 応用練習

```
void matrix_mult(int n, double A[N][N],
                 double B[N][N], double C[N][N])
{
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            C[i][j] = 0.0;
            for (int k = 0; k < n; k++)
                C[i][j] += A[i][k] * B[k][j];
        }
    }
}
```

上記の積は出力配列が入力配列と別領域であることを前提とする。　　が転置、
　　が行列積。

「分かっていたのに　点」を防ぐ

書く前に確認する　つの契約

関数名・引数型・順序・戻り値は配布ヘッダと一致するか。
自分が書くのは関数だけか、　　を含むプログラムか。
入出力は誰が担当するか。指定されない表示を加えない。
使用禁止・使用必須の関数、追加配列の禁止はあるか。
保存名と保存先、リンクする全ファイルは何か。

典型ミス　直し方

・　ではなく　へ数値を代入　指す先に書くなら　。
・　。
・　に　で　へ。
・カウンタを初期化しない　加算前に対象範囲を　へ。
・転置を全要素で交換　から。
・足し込みの初期化を内側ループに置く　加算開始前に　回。
・上限を例の　で固定　引数　。
・　の　だけでなく　も確認。
・配布　を含むのに　を追加　は　つ。
・入力案内を　へ。

が見逃さないもの

数字の違い、小数桁の違い、記号の欠落、大文字小文字、行の結合、余計な文字列。　は行内の空白差を無視し、　は空行だけの差を無視する。

隠しテストに備える共通セット

最小サイズ　/ 最大許容サイズ /　/ 負数が許されるなら負数 / 同値 / 逆順 /　回連続呼出し / 出力配列に初期値　を入れる。問題の前提内で試す。

期待値をハードコードしない。例の入力ではなく、仕様全体を実装する。合う出力を作るのではなく、正しい関数を書く。

即答ドリル　問

制限　分。まず解答を見ずに、理由を　行ずつ書く。

の後の　は？

の中から　へ　を渡す呼び出しは？

で　を読む書式と引数は？

のメンバー　を　で読む引数は？

に安全な最大幅付き　入力を書く。

が　内にある。　は？

関数　内でも同じ式で　を得られるか？

個のサイコロの和を　へ数える。最低何要素必要？

行列　をその場で転置すると？

転置の交換を全　で実行すると何が起きる？

の　は？

で　のリンクエラー。何を追加する？

にデバッグ文が混ざった。どこへ出すべき？

採点：各　点。　点以上なら模試へ。　点以下なら該当する基本ページを読み直し、別の数値で同じ問いを作る。

即答ドリルの解答・解説

。 は の番地ではなく、 が指している の値。

。 はすでに 。 は を渡すことになる。

。 では で を表示する。

。 の型が なら 。

。 番目の場所は終端文字用。

の配列なら 。仮引数 は へ調整されるので、関数内では配列全体のサイズは求まらない。長さ を別引数で渡す。

要素。添字 まで使うので、 〜 が必要。使わない 〜 も配列には含まれる。
なら 要素だが、指定の契約が変わる。

。対角線は不変で、 と を交換する。

非対角要素を 回ずつ交換して元に戻る。 から始める。

。 、 。

。宣言のため も必要。

。 。本番提出では不要なデバッグ出力を削除するのが安全。

説明できたら、次は白紙で書く

/ / / を、解答を閉じて各 分で書く。名前だけでなく、引数・初期化・終了条件・ヘッダも再現する。

模試　：問　・問

本書のオリジナル模試。各　点。入力は仕様を満たすものとする。各関数を指定　　へ書き、小さな　で呼び出して例と境界値を検証する。解答は　　〜　　ページ。

問　：秒を時・分・秒へ【型　】

　　に次の関数を作る。　は　以上の秒数。時を　　、分を　　、秒を　　に格納する。分・秒は　〜　　。関数内で入出力しない。

```
/* hms.h */
void hms(int t, int *h, int *m, int *s);

/* Test call */
int h, m, s;
hms(3661, &h, &m, &s);
printf("%d:%02d:%02d\n", h, m, s);
```

検証例：　　/　　。

追加条件：　、　　、　　、　　でも正しく動作すること。

問　：三角形の面積と重心【型　】

　　に　関数を作る。　　は以下の型。面積は絶対値をとり、重心は　の指す変数に書き込む。頂点が一直線なら面積　。

```
/* tri.h */
typedef struct { double x, y; } point_t;
double tri_area(point_t *a, point_t *b, point_t *c);
void centroid(point_t *a, point_t *b,
              point_t *c, point_t *g);
```

面積 =　　。

重心 =　　。

検証例：　　面積　、重心　　。

頂点　　を入れ替えても面積　。

採点の目安：各問とも例の一致だけでなく、全仕様・指定の関数型・追加入力を満たしたとき　点。自己採点用であり公式の採点を保証するものではない。

模試　：問　・問

問　：　個のサイコロの和の分布【型　】

に　関数を書く。　は　を順に　回呼んで配列に格納。　は和　の回数を　に記録する。　〜　。　は　要素以上。　は呼ばない。

```
void dice_throw(int d[], int n);
void dice2_distr(int n, int d1[], int d2[], int count[]);
```

分布の検証では乱数を使わず、　、　を渡す。結果は　、　、　、　、ほかの　〜　は　。呼出前に　へ　を入れても同じこと。

乱数列そのものを手で予想する必要はない。発生関数は各要素が　〜　であることを確認する。　なら　〜　がすべて　。

問　：行列積とトレース【型　・応用】

　とする。　に　と対角成分の和を返す関数を書く。　。　は　と異なる領域。計算関数内の入出力は禁止。

```
#define N 4
void matrix_mult(int n, double A[N][N],
                 double B[N][N], double C[N][N]);
double matrix_trace(int n, double A[N][N]);
```

検証例：　、　、　。　、　。　を　で埋めてから呼んでも同じ結果。　で　なら　、　。

残り　　分の確認

- 　を余分に作っていない
- 使う　をすべてリンクした
- 数式関数には　と　を指定
- 配列の初期化とループ上限を確認
- 例以外を少なくとも　件実行
- 保存名・保存場所を確認

模試　：問　・問　の解答

問

```
#include "hms.h"
void hms(int t, int *h, int *m, int *s)
{
    *h = t / 3600;
    *m = (t % 3600) / 60;
    *s = t % 60;
}
```

だけだと分が　以上になる。秒も　ではなく　。出力は　の仕事。

問

```
#include <math.h>
#include "tri.h"
double tri_area(point_t *a, point_t *b, point_t *c)
{
    double z = (b->x - a->x) * (c->y - a->y)
             - (c->x - a->x) * (b->y - a->y);
    return fabs(z) / 2.0;
}
void centroid(point_t *a, point_t *b,
              point_t *c, point_t *g)
{
    g->x = (a->x + b->x + c->x) / 3.0;
    g->y = (a->y + b->y + c->y) / 3.0;
}
```

面積の符号は頂点の順序で変わるため　が必要。重心は　というポインタそのものではなく、
と　へ書き込む。

リンクと境界値

```
gcc -Wall -Wextra hmsmain.c hms.c -o hmsrun
gcc -Wall -Wextra trimain.c tri.c -lm -o trirun
```

ファイル名は自作したものに合わせる。　は　。三角形は逆順・同一点・一直線・負の座標を試す。

模試　：問　・問　の解答

問

```
#include <stdlib.h>
void dice_throw(int d[], int n)
{
    for (int i = 0; i < n; i++) d[i] = rand() % 6 + 1;
}
void dice2_distr(int n, int d1[], int d2[], int count[])
{
    for (int s = 2; s <= 12; s++) count[s] = 0;
    for (int i = 0; i < n; i++)
        count[d1[i] + d2[i]]++;
}
```

実際にヘッダを使うときはその　　も追加。　　を　にするループを入力データのループ内へ入れると、過去の集計が消える。

問

```
#include "mat.h"
void matrix_mult(int n, double A[N][N],
                 double B[N][N], double C[N][N])
{
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            C[i][j] = 0.0;
            for (int k = 0; k < n; k++)
                C[i][j] += A[i][k] * B[k][j];
        }
    }
}
double matrix_trace(int n, double A[N][N])
{
    double sum = 0.0;
    for (int i = 0; i < n; i++) sum += A[i][i];
    return sum;
}
```

行列積は　の　行と　の　列の内積。行列ベクトル積と違い、出力　　ごとに初期化する。転置や要素ごとの積とは別の演算。

模試 ：問 ・問

模試 から時間を空け、白紙から解く。各 点、合計 点。型は同じでも、条件・戻り値・境界値を必ず読み直す。解答は 〜 ページ。

問 ： 個の整数を昇順へ【型 】

に と を実装する。 が戻ったとき にする。 の中で を使うこと。入出力は禁止。

```
void swap_int(int *a, int *b);
void sort3(int *a, int *b, int *c);
```

検証例： / 。
個の大小関係の全 通り、同値、負数も試す。
ポイント： の引数 はすでに番地を保持している。

問 ：構造体の読み込みと割引【型 】

に 関数を作る。 は品名・単価・個数を読む。成功 、読込み前の なら 、不正データなら 。割引後の単価は整数演算で 。 は 〜 。

```
typedef struct {
    char name[16];
    int price;
    int qty;
} item_t;
int item_scan(item_t *p);
void item_discount(item_t *p, int pct);
int item_total(item_t *p);
```

検証例：入力 、 、合計 。
入力 、 、合計 。
名前は空白を含まない バイト以内、単価・個数は非負で計算結果が の範囲に収まるものとする。

模試 ：問 ・問

問 ：配列の最長連続区間【型 】

に関数 を書く。配列 の中で同じ値が連続する区間の長さの最大を返す。 、 なら 。配列を書き換えない。

```
int longest_run(int n, int a[]);
```

検証例： 。
、 、要素なし 。
最後で終わる区間も忘れず反映すること。

問 ：対称行列の判定と行交換【型 】

に 関数を書く。 、 。
は が全要素で成立すれば 、そうでなければ を返す。今回は問題仕様として完全一致で比較する。

は の行を交換する。新しい配列は使わず、 の一時変数だけを用いる。 なら値は変わらない。

```
#define N 4
int matrix_symmetric(int n, double A[N][N]);
void matrix_swap_rows(int n, double A[N][N],
                      int r, int s);
```

対称性の検証： / 。
行交換の検証： の 行と 行 。 や同一行の交換でも範囲外に触れない。

自己採点と復習記録

問 ： 問 ： 問 ： 問 ：
かかった時間： 分 合計：
原因を分類： 仕様読解 型・番地 初期化 添字 検証不足
次に白紙で再現する関数：

> 同じ問題を覚えただけでは不十分。値やサイズを変える、入力順を逆にする、関数を 回呼ぶなどして「型を理解したか」を確かめる。

模試　：問　・問　の解答

問

```
void swap_int(int *a, int *b)
{
    int t = *a;
    *a = *b;
    *b = t;
}
void sort3(int *a, int *b, int *c)
{
    if (*a > *b) swap_int(a, b);
    if (*b > *c) swap_int(b, c);
    if (*a > *b) swap_int(a, b);
}
```

最後の　　の比較が必要。　　では最初の　回だけだと　　になる。

問

```
#include <stdio.h>
#include <stdlib.h>
#include "item.h"
int item_scan(item_t *p)
{
    int r = scanf("%15s%d%d", p->name,
                  &p->price, &p->qty);
    if (r == EOF) return EOF;
    if (r != 3) exit(1);
    return 1;
}
void item_discount(item_t *p, int pct)
{
    p->price = p->price * (100 - pct) / 100;
}
int item_total(item_t *p)
{
    return p->price * p->qty;
}
```

例の　　は　　。先に　　を計算してから　　を掛けると結果が変わる。今回は中間計算も　　範囲内という前提。

模試　：問　・問　の解答

問

```
int longest_run(int n, int a[])
{
    if (n == 0) return 0;
    int best = 1;
    int len = 1;
    for (int i = 1; i < n; i++) {
        if (a[i] == a[i - 1]) len++;
        else len = 1;
        if (len > best) best = len;
    }
    return best;
}
```

要素数　なら　　に触れる前に戻る。　　を毎回更新するため、最後の区間も集計できる。

問

```
#include "mat.h"
int matrix_symmetric(int n, double A[N][N])
{
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if (A[i][j] != A[j][i]) return 0;
    return 1;
}
void matrix_swap_rows(int n, double A[N][N],
                      int r, int s)
{
    for (int j = 0; j < n; j++) {
        double t = A[r][j];
        A[r][j] = A[s][j];
        A[s][j] = t;
    }
}
```

対称性判定は片側だけで十分。　　では比較ループが　回も実行されず　を返す。一般の実数計算では誤差許容が必要な場合もあるが、本問は完全一致という指定を守る。

補講：動的割当ての原理

ここからは添付　資料の補講。　本編とは分け、試験範囲に含まれる場合や以後の授業の復習に使う。

確保　失敗確認　使用　解放

```
#include <stdio.h>
#include <stdlib.h>

int n = 6;                  /* n > 0 */
int *a = malloc(sizeof *a * (size_t)n);
if (a == NULL) {
    fprintf(stderr, "malloc failed\n");
    exit(1);
}
for (int i = 0; i < n; i++) a[i] = 23;
free(a);
a = NULL;
```

は指定バイト数の領域へのポインタを返す。内容は未初期化。　は要素　個の大きさであり、　自体のポインタサイズとは違う。　では　の返り値のキャストは不要。

資料を読むときの重要な補正

- は演算子。型名には括弧が必要：　。
　は不正。変数なら　または　。
- の結果は　型。表示は　。
- が　バイト、ポインタが　バイトとは限らない。環境依存。
- はポインタ変数を　にしない。自分で代入する。
- 別名ポインタが残る場合、　だけで全体が安全にはならない。

解放済み領域の使用 / 二重　/ 配列の範囲外 / 確保量の不足は未定義動作。　は何もしない。通常のローカル配列を　してはいけない。

補講：文字列と行の逆順

長さ の文字列に必要なのは バイト

```
char *strdouble(char s[])
{
    size_t len = strlen(s);
    char *t = malloc(2 * len + 1);
    if (t == NULL) exit(1);
    for (size_t i = 0; i < len; i++) {
        t[i] = s[i];
        t[len + i] = s[i];
    }
    t[2 * len] = '\0';
    return t;
}
```

と を する。課題 では ・ は禁止。上記は が で表せる前提。非常に大きい入力では確保サイズのオーバーフローも検査する。

行を逆順に出す構造

はポインタの配列。毎回同じ読込バッファの番地を格納すると、最後の文字列ばかりになる。各行専用の領域を確保し、コピーする必要がある。

```
/* after fgets(buf, sizeof buf, stdin) succeeds */
line[n] = malloc(strlen(buf) + 1);
if (line[n] == NULL) exit(1);
strcpy(line[n], buf);
n++;

/* after all lines have been stored */
for (int i = n - 1; i >= 0; i--) fputs(line[i], stdout);
for (int i = 0; i < n; i++) free(line[i]);
```

断片コード。 が上限を超える前に検査し、エラー時も既存行を解放する。 は改行も読み込み、 等では を返す。長い 行がバッファで分割される点にも注意する。

を通常の配列で宣言した場合、 はしない。動的に確保した を それぞれ解放する。最終行に改行がない場合の出力方針も考える。

補講：構造体・スタック

構造体本体と内部の配列は別々の領域

```
typedef struct {
    int size;
    int sp;
    int *data;
} istack_t;

/* s and s->data were allocated separately */
void istack_delete(istack_t *s)
{
    if (s == NULL) return;
    free(s->data);
    free(s);
}
```

先に　　すると　　を取り出すこと自体が不正になる。呼出側の　を　　にしたければ、　　と書く。関数内でローカルな　を　　にしても呼出側は変わらない。

は現在のデータ数 / 次の格納位置

```
int istack_empty(istack_t *s) { return s->sp == 0; }
int istack_full(istack_t *s) { return s->sp == s->size; }

/* fixed-capacity version */
void istack_push(istack_t *s, int d)
{
    if (istack_full(s)) return;
    s->data[s->sp++] = d;
}
int istack_pop(istack_t *s)
{
    if (istack_empty(s)) return 0;
    return s->data[--s->sp];
}
```

　　は格納後に　　を増やし、　　は先に　　を減らして読む。空の　　で　を返すのは課題の仕様であり、値　が実際に入っていた場合と戻り値だけでは区別できない。

課題　　の前半は満杯なら無視。後半は満杯なら　　で　倍。　つの仕様を同時に混ぜない。拡張後の　　と　　の更新を忘れない。

補講：得点に直結する小問

必要バイト数

、 と仮定する。
要素 要素 のコピー
を 回つなぐ文字列 を 回つなぐ文字列

間違いを つ指摘

```
int *a = malloc(sizeof(a) * n);
a[n] = 7;
free(a);
printf("%d\n", a[0]);
```

の安全な受け方

次を直す。確保失敗時に元のポインタを失わず、成功時だけ を更新する。元の は正で、倍の値と必要バイト数が表現可能であると仮定する。

```
s->size *= 2;
s->data = realloc(s->data, sizeof(int) * s->size);
```

スタックの追跡

初期容量 、空。満杯時に容量を 倍へ拡張する。

。

各操作後の 、 、保持するデータ列と、 の戻り値を書く。

度数分布の添字

入力配列 を、 要素の配列で集計する。
、 、必要要素数、 を記録する添字を答える。
、 のとき、何要素必要か。

所有権と解放

の各要素を で確保した。何回 が必要か。 自身も すべきか。 した に が含まれるとき、 だけも すべきか。

補講：小問の解答

-

: 、 、 、 、 バイト。空文字列にも終端文字 バイトが必要。

: はポインタのサイズ / は範囲外 / 後に を使っている。さらに の失敗確認も必要。

一時ポインタを使う

```
int new_size = s->size * 2;
int *tmp = realloc(s->data, sizeof *tmp * new_size);
if (tmp == NULL) {
    fprintf(stderr, "realloc failed\n");
    /* original s->data is still valid here */
    free(s->data);
    free(s);
    exit(1);
}
s->data = tmp;
s->size = new_size;
```

・ が必要。上記は失敗時に解放して終了する方針。 成功後、元のポインタは使わない。実用コードでは倍増計算のオーバーフローも事前に確認する。

スタック追跡

```
operation   size  sp  data       return
push(5)       2    1  [5]        -
push(6)       2    2  [5,6]      -
pop()         2    1  [5]        6
push(9)       2    2  [5,9]      -
push(8)       4    3  [5,9,8]    -
pop()         4    2  [5,9]      8
```

-

: 、 、 要素。添字は値 なので、 、 、 。大きな値の例も差は なので 要素。 の引き算が であふれる入力では、先に広い型へ変換する。

: 各 を 回ずつ、合計 回。通常の配列 自身は しない。構造体内の固定配列 は構造体と一体なので、別途 しない。

最終チェックと復習記録

前日：何も見ずに再現する　項目

/ / のポインタ引数
の と 、距離の計算
構造体メンバーの入力・変更・表示
と度数カウントの　初期化
行列の読み込み・和・別配列への転置
同じ配列内での転置・行列ベクトル積
リダイレクト

当日：最後の　分

指定フォルダ・指定ファイル名か。 になっていないか。
保存後のコードでコンパイルしたか。
配布ヘッダと型・関数名が一致するか。
を増やしていないか。必要 が全部リンクされているか。
に入力案内やデバッグ文を混ぜていないか。
、同値、負数など前提内の境界値を試したか。
初期化・ループ条件・ ・添字を再確認したか。
試験環境のログアウト等は勝手にせず、教員指示に従ったか。

振り返り欄

模試　： 点 / 分　解き直し日：
模試　： 点 / 分　解き直し日：
一番多いミス：
次回の最初に確認すること：

資料と検証について

参照：既存アプリの 対策（ ・ ）および添付 課題・講義 。動画内容の独立検証は行っていない。本編の模試は独自作成。掲載した模試解答と は、生成時に でコンパイルし、代表値・境界値の自動テストを実施。

最後は「読める」ではなく「書ける」で判断。苦手な　関数を、閉じる　書く　動かす　比べる。これを繰り返すのが最短ルート。